KNN Fine-Tuning, SMOTE, and Cost Sweep

Conversation notes and code examples

This document organizes the conversation into a practical guide. Code examples assume a binary classification task and that X and y have been prepared.

1. Using GridSearchCV to fine-tune KNN

GridSearchCV tests hyperparameter combinations using cross-validation and selects the combination with the best average validation score.

Common KNN parameters include n_neighbors (number of neighbors), weights (uniform or distance), metric (distance definition), and p (Minkowski distance power).

Scale numeric features before KNN because distance calculations are sensitive to feature scales.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 11, 15],
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["euclidean", "manhattan"],
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    scoring="accuracy",  # Consider f1, f1_macro, or balanced_accuracy for imbalance
    cv=cv,
    n_jobs=-1,
    return_train_score=True
)

grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_

print("Best parameters:", grid_search.best_params_)
print("Best CV score:", grid_search.best_score_)
```

2. Building a SMOTE + KNN pipeline

Use imblearn.pipeline.Pipeline so SMOTE runs only on each training fold during cross-validation. Do not run SMOTE on the full dataset before splitting.

The example below assumes X contains numeric, already encoded features. For mixed numerical/categorical data, consider SMOTENC and a suitable preprocessing design rather than applying ordinary SMOTE blindly to one-hot encoded columns.

sampling_strategy=1.0 targets equal minority and majority class counts; 0.5 targets a minority-to-majority ratio of 1:2. The SMOTE k_neighbors parameter is separate from the classifier's n_neighbors.

```
import numpy as np
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("knn", KNeighborsClassifier())
])

param_grid = {
    "smote__sampling_strategy": [0.5, 0.75, 1.0],
    "smote__k_neighbors": [3, 5],
    "knn__n_neighbors": [3, 5, 7, 11, 15],
    "knn__weights": ["uniform", "distance"],
    "knn__p": [1, 2]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    scoring="average_precision",
    cv=cv,
    n_jobs=-1,
    refit=True
)

grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_

print("Best parameters:", grid_search.best_params_)
print("Best CV average precision:", grid_search.best_score_)
```

3. Evaluate the tuned model

For a binary classifier, the confusion matrix contains true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP). Average precision can be informative when the positive class is rare.

```
from sklearn.metrics import (
    classification_report, confusion_matrix,
    balanced_accuracy_score, average_precision_score
)
```

```
y_pred = best_model.predict(X_test)
y_prob = best_model.predict_proba(X_test)[:, 1]

print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred))
print("Average precision:", average_precision_score(y_test, y_prob))
```

4. Cost sweep over thresholds and error costs

A cost sweep assigns costs to false positives and false negatives. Total cost = C_FP × FP + C_FN × FN. The example keeps the false-positive cost at 1 and varies the false-negative cost.

The positive class must be encoded as 1 for this example. If labels are strings, encode them consistently first.

For a rigorous evaluation, choose the threshold on a validation set or using cross-validation predictions, then evaluate the chosen threshold once on the untouched test set.

```
from sklearn.metrics import confusion_matrix
import pandas as pd
import numpy as np

cost_ratios = [1, 2, 3, 5, 10]
thresholds = np.arange(0.05, 0.96, 0.05)
results = []

for cost_fn in cost_ratios:
    cost_fp = 1
    for threshold in thresholds:
        y_pred_threshold = (y_prob >= threshold).astype(int)
        tn, fp, fn, tp = confusion_matrix(
            y_test, y_pred_threshold, labels=[0, 1]
        ).ravel()

        total_cost = cost_fp * fp + cost_fn * fn
        results.append({
            "cost_fp": cost_fp,
            "cost_fn": cost_fn,
            "threshold": threshold,
            "TN": tn, "FP": fp, "FN": fn, "TP": tp,
            "total_cost": total_cost,
            "cost_per_sample": total_cost / len(y_test)
        })

cost_df = pd.DataFrame(results)

# Lowest-cost threshold for each cost scenario
best_cost_results = (
    cost_df.sort_values("total_cost")
           .groupby(["cost_fp", "cost_fn"], as_index=False)
           .first()
)
print(best_cost_results.to_string(index=False))
```

5. Plot total cost versus threshold

Each line represents a different false-negative cost. The minimum point on a line is the lowest observed cost among the tested thresholds for that scenario.

```
import matplotlib.pyplot as plt

for cost_fn in sorted(cost_df["cost_fn"].unique()):
    subset = cost_df[cost_df["cost_fn"] == cost_fn]
    plt.plot(
        subset["threshold"], subset["total_cost"],
        marker="o", label=f"FN cost = {cost_fn}"
    )

plt.xlabel("Classification Threshold")
plt.ylabel("Total Misclassification Cost")
plt.title("Cost Sweep: Total Cost vs. Threshold")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

6. Plot false positives versus false negatives

This plot shows the trade-off between false positives and false negatives at a selected false-negative cost.

```
cost_fn = 5
subset = cost_df[cost_df["cost_fn"] == cost_fn]

plt.plot(subset["threshold"], subset["FP"], marker="o", label="False Positives")
plt.plot(subset["threshold"], subset["FN"], marker="s", label="False Negatives")
plt.xlabel("Classification Threshold")
plt.ylabel("Number of Errors")
plt.title(f"FP vs. FN (FN Cost = {cost_fn})")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

7. Plot average cost per sample

```
for cost_fn in sorted(cost_df["cost_fn"].unique()):
    subset = cost_df[cost_df["cost_fn"] == cost_fn]
    plt.plot(
        subset["threshold"], subset["cost_per_sample"],
        marker="o", label=f"FN cost = {cost_fn}"
    )

plt.xlabel("Classification Threshold")
plt.ylabel("Average Cost per Sample")
plt.title("Average Misclassification Cost by Threshold")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

8. Calculate and plot precision, recall, and F1

These metrics describe positive-class performance at each threshold. Use validation predictions for threshold selection to avoid tuning on the final test set.

```
from sklearn.metrics import precision_score, recall_score, f1_score

metric_results = []
for threshold in sorted(cost_df["threshold"].unique()):
    y_pred_threshold = (y_prob >= threshold).astype(int)
    metric_results.append({
        "threshold": threshold,
        "precision": precision_score(y_test, y_pred_threshold, zero_division=0),
        "recall": recall_score(y_test, y_pred_threshold, zero_division=0),
        "f1": f1_score(y_test, y_pred_threshold, zero_division=0)
    })

metrics_df = pd.DataFrame(metric_results)

for metric in ["precision", "recall", "f1"]:
    plt.plot(metrics_df["threshold"], metrics_df[metric],
             marker="o", label=metric.capitalize())

plt.xlabel("Classification Threshold")
plt.ylabel("Metric Score")
plt.title("Precision, Recall, and F1 vs. Threshold")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

9. Recommended experimental comparison

Compare baseline KNN without SMOTE, SMOTE + KNN, and threshold adjustment using the same splits and cost assumptions.

Report accuracy, balanced accuracy, precision, recall, F1, false positives, false negatives, total cost, average cost per sample, and the selected threshold.

SMOTE can affect probability calibration. Treat predict_proba outputs as scores unless calibration has been checked. For categorical features, consider SMOTENC or compare with class weighting and random oversampling.

Conversation questions covered

- How to use GridSearchCV for fine-tuning KNN models
- How to build a SMOTE pipeline and evaluate it with a cost sweep
- How to plot metric results from cost_df